Do We Need the Shadow?

Freezing-Aware Weight Versioning for Pipeline-Parallel Training

Animesh Jain Rupankar Podder

CS6886 Systems Engineering for Deep Learning Code: `https://github.com/2coolcoder/ShadowFRZ`

1 Introduction

When a model is split across d GPUs, plain backpropagation keeps all but one GPU idle: each stage waits for the gradient from the stage after it. Two families of methods remove this wait. *Decoupled backpropagation* (DDG [1]) lets every module use a stale error gradient, so all modules run their backward passes at once. *Pipeline parallelism* (GPipe [4], PipeDream [2]) keeps m microbatches in flight. Then the weights change while a microbatch is between its forward and backward pass, so PipeDream stores up to d weight versions per stage. PipeDream-2BW [3] cuts this to two: the current weights and a *shadow* copy. The price is a uniform one-step delay, $W^{(t+1)} = W^{(t)} - \nu\nabla f(W^{(t-1)})$.

A separate line of work, *layer freezing* (FreezeOut [5], PolicyFRZ [6]), stops updating layers that have converged to save compute. Our idea was to merge the two. Per-layer gradient statistics would decide where 2BW's shadow is still needed: still-changing layers keep it, slowly changing ones drop it, and converged ones are frozen. This should save memory without losing accuracy. We asked five questions: (1) Does DDG still match backprop against a modern baseline? (2) Does BERT need 2BW's shadow at all? (3) Does the merge beat 2BW and PolicyFRZ on the models each was evaluated on? (4) Which component gives which gain, and is the freezing signal better than chance? (5) How do the effects scale from BERT-Mini to BERT-Large?

2 Prior work

DDG [1] splits a network into K modules and gives module k error gradients that are $K-k$ steps old. It reports backprop-level accuracy on CIFAR ResNets, but its backprop baseline uses an older training recipe. **PipeDream** [2] introduced the one-forward-one-backward (1F1B) schedule and weight stashing. **PipeDream-2BW** [3] double-buffers the weights, coalesces gradients over m microbatches, and reuses the hyperparameters of synchronous training. Its paper also describes **PipeDream-Flush**, which keeps one weight version and flushes the pipeline every batch; this is the same semantics as GPipe [4]. **FreezeOut** [5] freezes layers on a fixed schedule. **PolicyFRZ** [6] freezes a layer when its gradient variation (GV) and relative magnitude (AM) both fall below thresholds. It reports ∼28% compute savings on ResNet-50 against a 92.0% CIFAR-10 baseline. Related systems drop weight stashing with learning-rate corrections (PipeMare [7]) or remove frozen layers from a GPipe pipeline (PipeTransformer [8]). Neither studies how freezing interacts with 2BW's two weight versions.

For models we use CIFAR ResNets [9] with a modern recipe [10], and BERT [11] in the compact sizes of [12] with the Pre-LN layout of Megatron-LM [13].

3 Idea and implementation

Variants. All variants share one 1F1B engine and differ only in which weight version each stage uses. We compare **vanilla** (Flush semantics, 1 copy, no staleness) and **2BW** (2 copies) with: **1BW**, i.e. 2BW with the shadow removed (1 copy); **FRZ**, the PolicyFRZ rule alone; **2BW-Sel**, which drops the shadow per layer where AM is low; **2BW+FRZ**; and **Full**, our merge (Sel + FRZ). Three controls test whether the signal matters: freezing random layers on the same budget, a fixed FreezeOut schedule, and resetting Adam's state when a layer is unfrozen. In 1BW the update is applied in place, so only microbatches that are in flight during an update see different weights in forward and backward. On stage s this is a fraction $(d-s)/m$ of microbatches; all others are fresh.

Freezing rule. Over windows of 100 steps we compute GV and AM per layer. A layer is frozen if $\mathrm{GV}<\theta_1$ and $\mathrm{AM}/\mathrm{AM}_{\mathrm{ref}}<\theta_2$, and its shadow is dropped if $\mathrm{AM}/\mathrm{AM}_{\mathrm{ref}}<0.8$. Decisions are made only at weight-version boundaries and not in the first 10% of training. Frozen layers are thawed and re-measured every 4 windows. We set the thresholds once, from the vanilla runs, before any freezing run: θ_1=0.3, with θ_2=0.5 for ResNet and 0.6 for BERT.

Exact emulation on 2 GPUs. The weight version a stage uses depends only on $(d, m, \text{schedule})$, not on the number of devices. Our `pipeemu` library builds a version table $(s,k) \mapsto (v_F, v_B)$ from a 1F1B simulator. It then runs the whole d-stage pipeline on one GPU, copying version v_F into the weights in place before each forward pass and v_B before each backward pass. This requires fp32/TF32, since mixed precision would hide the mismatch. Three checks validate the emulator:

- vanilla equals gradient accumulation exactly;
- 2BW matches its closed-form update to within 6×10^{-17};
- a real 2-GPU pipeline (`torchrun`) matches the emulator bit for bit for Flush, 2BW and 1BW, and its measured per-stage memory matches our memory model.

Memory for d>2 is modelled from per-layer profiles. Each stage stores versions · W + grads + optimizer state + $(d-s+1)$ · activations. Time per batch comes from the 1F1B simulator, with fixed partitions for each model.

Experiments. All runs use d=m=4 unless noted (≈44 h on two GPUs). **Track A:** DDG with $K \in \{1, 2, 4\}$ on ResNet-56/110, CIFAR-10/100 (17 runs).

ResNet-50, C10	Acc. (%)	Δ van.	Mem.[a]	Saved[b]
Vanilla (2 seeds)	95.62±.09	–	−0.1%	–
2BW (2 seeds)	95.39±.01	−0.23	–	–
1BW (2 seeds)	**95.84**±.14	+0.22	−0.1%	–
FRZ only	95.81	+0.19	−9.0%	4.0%
2BW-Sel	95.14	−0.48	0%	–
Full (ours) (2 seeds)	95.52±.14	−0.10	−8.6%	3.6–4.6%
Full, random layers	95.51	−0.11	−8.5%	3.9%
Full, FreezeOut sched.	95.34	−0.28	−46.3%	18.7%
Full, aggressive thr.	94.68	−0.94	–	8.4%

Table 1: Track B. [a]Average stage-1 memory vs. 2BW (model). [b]Backward compute saved by freezing. PolicyFRZ reports 92.0% and ∼28% savings.

MLM val. loss (lr)	Vanilla	2BW	**1BW**	Full
Mini (1e-3)	3.250	3.492	**3.303**	3.509
Small (5e-4)	2.859	3.010	**2.914**	3.020
Medium (5e-4)	2.770	3.001	**2.886**	3.029
Base (1e-4)	2.982	3.093	–	3.104

Table 2: Track C, Pre-LN BERT pre-training, 20k steps. Medium vanilla, 2BW and Full are 2-seed means (seed noise ±0.03).

Track B: ResNet-50 on CIFAR-10 for 160 epochs, PolicyFRZ's setting (18 runs). **Track C:** Pre-LN BERT Mini/Small/Medium/Base, masked-language-model (MLM) pre-training on WikiText-103 [14] for 20k steps (31 runs, plus 31 learning-rate sweep runs), and SST-2 fine-tuning from GLUE [15], from our checkpoints and from the public `bert-base`/`bert-large` checkpoints (36 runs). All variants reuse the vanilla hyperparameters, as 2BW prescribes.

4 Evaluation against the baselines

DDG vs. modern backprop (Q1). With our modern recipe, backprop reaches 94.22% on ResNet-56/CIFAR-10, against 93.12% in the DDG paper. With DDG's own recipe we get 92.81%, so the weaker baseline comes entirely from the recipe. K=2 stays within 0.1–0.25 points of backprop on CIFAR-10 but loses 0.57 on ResNet-56/CIFAR-100. K=4 is below backprop in every configuration, by 0.13 to 1.62 points.

The shadow is what makes 2BW stale (Q2). On ResNet-50 (Table 1), 2BW loses 16 points of accuracy at epoch 4 and still trails vanilla by 0.23 at the end. 1BW tracks vanilla from the first epoch and ends at 95.84%. In BERT pre-training (Table 2, Fig. 1), 1BW closes 78%, 64% and 50% of 2BW's loss gap for Mini, Small and Medium. On Medium at fixed global batch, the gap tracks the predicted mismatch fraction. At m/d=4 only 11% of microbatches are mismatched and 1BW is within 0.01 of vanilla. Dropping the shadow also saves 5–8% of stage-1 memory on BERT. Running 2BW for only the first 10% of training keeps most of the gap: the early phase decides the outcome. The delay also narrows the usable learning rate. At learning rates where vanilla trains, both 2BW and 1BW on Post-LN BERT-Mini stay stuck at a loss of 7.24; this is why we use Pre-LN.

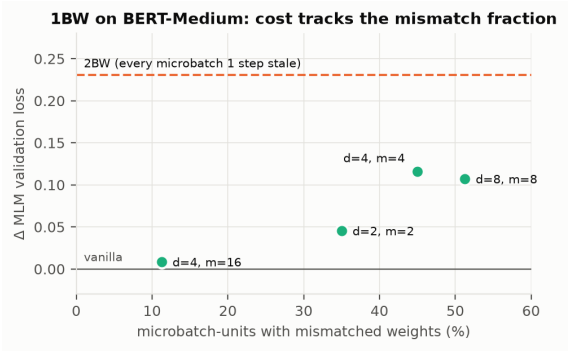


Figure 1: BERT-Medium, global batch 128. 1BW's loss gap to vanilla shrinks with the fraction of mismatched microbatches. 2BW's gap does not depend on d.

The merge vs. 2BW and PolicyFRZ (Q3, Q4). In pre-training, Full is within noise of 2BW: +0.13 points on ResNet-50 and +0.028 loss on BERT-Medium. The ablations explain why:

- Removing *all* shadows (1BW) gives the accuracy gain.
- Removing them *selectively* keeps the shadows early in training, which is exactly when they hurt.
- Freezing gives the memory gain (−8.6% stage-1 memory, −4% time per batch on ResNet-50).

On ResNet-50 the GV/AM signal does no better than freezing random layers on the same budget. On PolicyFRZ's own model, our baseline is 3.6 points stronger, and the rule saves ∼4% of compute rather than 28%. Forcing earlier freezing costs 0.94 points. During pre-training GV stays near the noise level for every layer, so "GV low and AM low" only holds in the low-learning-rate tail. Fine-tuning is different, because gradients shrink as the task is learned. On SST-2 from the public checkpoints the rule freezes 24–26% of layer-windows. Full reaches 94.38% on BERT-Large (2BW 93.81, vanilla 93.35) with 15.6% less stage-1 memory than 2BW (12.9% on Base, at 91.51 vs. 92.55).

Scaling (Q5). At a fixed learning rate, 2BW's gap grows with depth: +0.15 on Small, +0.23 on Medium. 1BW's advantage shrinks with more stages at fixed m/d and comes back with larger m/d.

Limitations. Memory and time for d>2 are modelled; the per-stage deltas are validated on the real 2-GPU pipeline. Most rows have one seed (two for key rows), so we do not interpret differences below seed noise. Pre-training covers only the first 20k steps.

Conclusion. 2BW's shadow is the source of its staleness. Removing it uses less memory and trains better, and with $m/d \geq 4$ it is close to exact. PolicyFRZ's statistics did not identify converged layers in pre-training against strong baselines, but they do when fine-tuning, where the merge saves 13–16% of memory at equal accuracy.

References

[1] Z. Huo, B. Gu, Q. Yang, H. Huang. Decoupled Parallel Backpropagation with Convergence Guarantee. *ICML*, 2018. `https://proceedings.mlr.press/v80/huo18a.html`

[2] D. Narayanan, A. Harlap, A. Phanishayee, V. Seshadri, N. R. Devanur, G. R. Ganger, P. B. Gibbons, M. Zaharia. PipeDream: Generalized Pipeline Parallelism for DNN Training. *SOSP*, 2019. https://doi.org/10.1145/3341301.3359646

[3] D. Narayanan, A. Phanishayee, K. Shi, X. Chen, M. Zaharia. Memory-Efficient Pipeline-Parallel DNN Training. *ICML*, 2021. https://proceedings.mlr.press/v139/narayanan21a.html

[4] Y. Huang et al. GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism. *NeurIPS*, 2019. https://arxiv.org/abs/1811.06965

[5] A. Brock, T. Lim, J. M. Ritchie, N. Weston. FreezeOut: Accelerate Training by Progressively Freezing Layers. arXiv:1706.04983, 2017. https://arxiv.org/abs/1706.04983

[6] L. Gao, S. Han, X. Wang, X. Wang, Z. Zou, L. Jiang. PolicyFRZ: Adaptive Layer Freezing via Policy Networks for Accelerating DNN Training in Edge Environments. *Intelligent and Converged Networks*, 7(3), 2026. https://ieeexplore.ieee.org/abstract/document/11701677

[7] B. Yang et al. PipeMare: Asynchronous Pipeline Parallel DNN Training. *MLSys*, 2021. https://arxiv.org/abs/1910.05124

[8] C. He, S. Li, M. Soltanolkotabi, S. Avestimehr. PipeTransformer: Automated Elastic Pipelining for Distributed Training of Transformers. *ICML*, 2021. https://arxiv.org/abs/2102.03161

[9] K. He, X. Zhang, S. Ren, J. Sun. Deep Residual Learning for Image Recognition. *CVPR*, 2016. https://arxiv.org/abs/1512.03385

[10] Y. Chen. PyTorch CIFAR models (ResNet-56 training recipe). https://github.com/chenyaofo/pytorch-cifar-models

[11] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *NAACL*, 2019. https://arxiv.org/abs/1810.04805

[12] I. Turc, M.-W. Chang, K. Lee, K. Toutanova. Well-Read Students Learn Better: On the Importance of Pre-training Compact Models. arXiv:1908.08962, 2019. https://arxiv.org/abs/1908.08962

[13] M. Shoeybi et al. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. arXiv:1909.08053, 2019. https://arxiv.org/abs/1909.08053

[14] S. Merity, C. Xiong, J. Bradbury, R. Socher. Pointer Sentinel Mixture Models (WikiText-103). *ICLR*, 2017. https://arxiv.org/abs/1609.07843

[15] A. Wang et al. GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding. *ICLR*, 2019. https://arxiv.org/abs/1804.07461

[16] A. Jain, R. Podder. ShadowFRZ: code, results and reproduction scripts for this project. https://github.com/2coolcoder/ShadowFRZ